MATH3881/5881: Statistical Machine Learning Theory

Week 2: Theory of Feedforward Neural Networks

Sarat Moka *UNSW, Sydney*

Key Topics

Books:

(A) *Mathematical Engineering of Deep Learning* book by Liquet, Moka, and Nazarathy (2024).

2.1 Feedforward Neural Networks

A **feedforward neural network** (FNN), also called a *fully connected network*, *multi-layer perceptron* (MLP), or *dense network*, defines a parametric function

$$f_\theta : \mathbb{R}^p \longrightarrow \mathbb{R}^q$$

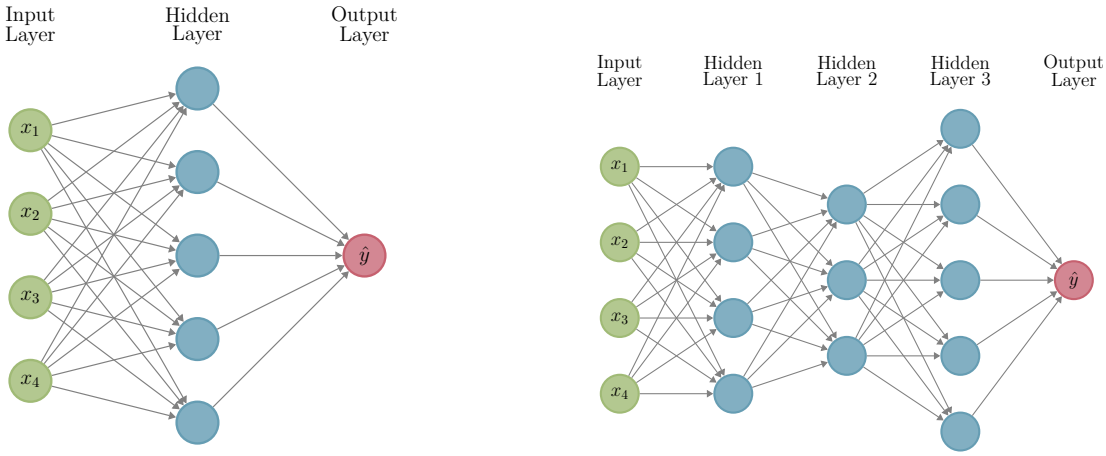
by composing a sequence of affine transformations and non-linear *activation functions*. The network takes a feature vector $x \in \mathbb{R}^p$ as input and produces an output $\hat{y} = f_\theta(x) \in \mathbb{R}^q$. The goal is to choose parameters θ so that f_θ approximates some target function f^*.

2.1.1 Architecture

A feedforward network with **depth** L consists of:

- an **input layer** with $N_0 = p$ units (the feature vector x, no computation),
- **hidden layers** $\ell = 1, \ldots, L-1$ with $N_1, \ldots, N_{L-1}$ neurons respectively,
- an **output layer** $\ell = L$ with $N_L = q$ neurons.

The integers $N_1, \ldots, N_{L-1}$ are called the **widths** of the hidden layers. The depth L counts hidden layers plus the output layer, but not the input layer. The total number of neurons (excluding input) is $\sum_{\ell=1}^{L} N_\ell$.



(a) A network with one hidden layer ($L = 2$). (b) A deep network with multiple hidden layers ($L = 4$).

Figure 2.1: Fully connected feedforward neural networks. Each circle is a neuron; each column of neurons is a layer.

2.1.2 The Forward Pass

At each layer ℓ, the network applies an affine transformation followed by an activation function. Writing $a^{[0]} = x$ and $\hat{y} = a^{[L]}$, the **forward pass** is

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]}, \qquad a^{[\ell]} = S^{[\ell]}\left(z^{[\ell]}\right), \qquad \ell = 1, \dots, L, \tag{2.1}$$

where:

- $W^{[\ell]} \in \mathbb{R}^{N_\ell \times N_{\ell-1}}$ is the **weight matrix** of layer ℓ,
- $b^{[\ell]} \in \mathbb{R}^{N_\ell}$ is the **bias vector** of layer ℓ,
- $S^{[\ell]} : \mathbb{R}^{N_\ell} \to \mathbb{R}^{N_\ell}$ is the **activation function** of layer ℓ.

For hidden layers $\ell = 1, \dots, L-1$, the activation $S^{[\ell]}$ is applied elementwise via a scalar function σ:

$$S^{[\ell]}(z) = \left[\sigma(z_1), \dots, \sigma(z_{N_\ell})\right]^\top. \tag{2.2}$$

For the output layer $\ell = L$, the activation depends on the task: sigmoid for binary classification, softmax for multi-class classification, or identity for regression (see Week 1).

The full network is therefore a composition of L such maps:

$$f_\theta(x) = S^{[L]}\big(W^{[L]} \cdots S^{[1]}\big(W^{[1]} x + b^{[1]}\big) \cdots + b^{[L]}\big). \tag{2.3}$$

Expanding layer by layer:

$$\begin{aligned}
&\text{Layer } 1 \begin{cases} z^{[1]} &= W^{[1]} \overbrace{x}^{\text{input}} + b^{[1]} \\ a^{[1]} &= S^{[1]}(z^{[1]}) \end{cases} \\
&\text{Layer } 2 \begin{cases} z^{[2]} &= W^{[2]}\, a^{[1]} + b^{[2]} \\ a^{[2]} &= S^{[2]}(z^{[2]}) \end{cases} \\
&\quad \vdots \\
&\text{Layer } L \begin{cases} z^{[L]} &= W^{[L]}\, a^{[L-1]} + b^{[L]} \\ \hat{y} = a^{[L]} &= S^{[L]}(z^{[L]}) \end{cases}
\end{aligned}$$

Remark 2.1.1

The vector $z^{[\ell]}$ is called the *pre-activation* and $a^{[\ell]}$ is the *activation* (or output) of layer ℓ. The hidden activations $a^{[1]}, \dots, a^{[L-1]}$ are sometimes called *learned features*, as they encode progressively abstract representations of the input.

2.1.3 Neuron-Level View

It is instructive to unpack equation (2.1) at the level of individual neurons. Let $w_{(i)}^{[\ell]}$ denote the i-th row of $W^{[\ell]}$ (a $1 \times N_{\ell-1}$ vector). Then the i-th neuron of layer ℓ computes

$$z_i^{[\ell]} = {w_{(i)}^{[\ell]}}^\top a^{[\ell-1]} + b_i^{[\ell]}, \qquad a_i^{[\ell]} = \sigma\left(z_i^{[\ell]}\right), \qquad i = 1, \ldots, N_\ell.$$

Each neuron takes a weighted sum of the previous layer's activations, adds a bias, and applies the scalar activation σ. This mimics the biological notion of a neuron firing when its total input exceeds a threshold.

2.1.4 Parameter Count

The trainable parameters of layer ℓ are the entries of $W^{[\ell]}$ and $b^{[\ell]}$, giving $N_\ell \times N_{\ell-1} + N_\ell = N_\ell(N_{\ell-1}+1)$ parameters per layer. The total number of parameters in the network is

$$|\theta| = \sum_{\ell=1}^{L} N_\ell(N_{\ell-1}+1). \tag{2.4}$$

Example 2.1.1

Consider the two networks in Figure 2.1.
(a) One hidden layer: $p = 4$, $N_1 = 5$, $q = N_2 = 1$. The parameters are:

$$W^{[1]} \in \mathbb{R}^{5\times 4}, \quad b^{[1]} \in \mathbb{R}^5, \quad W^{[2]} \in \mathbb{R}^{1\times 5}, \quad b^{[2]} \in \mathbb{R}^1,$$

giving $5 \cdot 5 + 1 \cdot 6 = 31$ parameters in total.
(b) Four layers: $p = 4$, $N_1 = 4$, $N_2 = 3$, $N_3 = 5$, $q = N_4 = 1$:

$$\underbrace{4 \cdot 5}_{\text{Layer 1}} + \underbrace{3 \cdot 5}_{\text{Layer 2}} + \underbrace{5 \cdot 4}_{\text{Layer 3}} + \underbrace{1 \cdot 6}_{\text{Layer 4}} = 20 + 15 + 20 + 6 = 61 \text{ parameters.}$$

Remark 2.1.2

The number of parameters grows quadratically in the width and linearly in the depth. Modern neural networks for image and language tasks routinely have millions to billions of parameters. Choosing the architecture (depth L, widths $N_1, \ldots, N_{L-1}$) is itself a design problem that significantly affects expressivity, generalization, and training difficulty.

Exercise 2.1.1

Consider a network with $p = 2$ inputs, one hidden layer of width $N_1 = 3$, and $q = 1$ output, with ReLU hidden activations and identity output activation.

(a) Write down the dimensions of all weight matrices and bias vectors, and verify the total parameter count using (2.4).

(b) Given

$$W^{[1]} = \begin{pmatrix} 1 & -1 \\ 0 & 2 \\ 1 & 0 \end{pmatrix}, \quad b^{[1]} = \mathbf{0}, \quad W^{[2]} = \begin{pmatrix} 1 & 1 & -1 \end{pmatrix}, \quad b^{[2]} = 0,$$

compute the output $f_\theta(x)$ for $x = (1,\, 1)^\top$.

(c) What is $f_\theta(x)$ for $x = (-2,\, 1)^\top$? Explain the role of the ReLU activations in this case.

2.2 Activation Functions

Recall from (2.1) that each hidden layer applies an elementwise scalar function $\sigma : \mathbb{R} \to \mathbb{R}$ to its pre-activation vector $z^{[\ell]}$. Without this non-linearity, the entire network would collapse to a single affine map regardless of depth, since compositions of affine functions are affine. The choice of σ has a significant impact on both the *expressivity* of the network and the *ease of training*.

2.2.1 The Main Activation Functions

Table 2.1 summarises the four activation functions used in practice for hidden layers, together with their derivatives $\dot\sigma$, which are required for the layer-wise chain rule used in backpropagation (made explicit in Week 3).

Activation	**Formula** $\sigma(u)$	**Derivative** $\dot\sigma(u)$
Sigmoid	$\dfrac{1}{1+e^{-u}}$	$\sigma(u)\big(1-\sigma(u)\big)$
Tanh	$\dfrac{e^{u}-e^{-u}}{e^{u}+e^{-u}}$	$1-\sigma_{\text{Tanh}}(u)^2$
ReLU	$\max(0,\, u)$	$\mathbf{1}\{u \geq 0\}$
Leaky ReLU	$\max(0,u) + \varepsilon \min(0,u)$	$\mathbf{1}\{u \geq 0\} + \varepsilon\, \mathbf{1}\{u < 0\}$

Table 2.1: Common scalar activation functions for hidden layers and their derivatives. Here $\varepsilon > 0$ is a small fixed constant (e.g. $\varepsilon = 0.01$).

Sigmoid. The sigmoid (or logistic) function maps any real number to $(0, 1)$:

$$\sigma_{\text{Sig}}(u) = \frac{1}{1 + e^{-u}}, \qquad \dot{\sigma}_{\text{Sig}}(u) = \sigma_{\text{Sig}}(u)\big(1 - \sigma_{\text{Sig}}(u)\big). \tag{2.5}$$

Its derivative is expressed conveniently in terms of the function itself. The sigmoid was historically the default hidden-layer activation and is still used as the output activation for binary classification.

Tanh. The hyperbolic tangent maps to $(-1, 1)$ and is zero-centred, which can be an advantage over sigmoid in hidden layers:

$$\sigma_{\text{Tanh}}(u) = \frac{e^{u} - e^{-u}}{e^{u} + e^{-u}}, \qquad \dot{\sigma}_{\text{Tanh}}(u) = 1 - \sigma_{\text{Tanh}}(u)^2. \tag{2.6}$$

Both sigmoid and tanh are **saturating**: as $|u| \to \infty$ the derivative $\dot{\sigma} \to 0$, meaning neurons transmit almost no gradient signal when their inputs are large in magnitude.

ReLU. The rectified linear unit is the dominant choice in modern deep networks:

$$\sigma_{\text{ReLU}}(u) = \max(0,\, u) = \begin{cases} 0 & u < 0 \\ u & u \geq 0, \end{cases} \qquad \dot{\sigma}_{\text{ReLU}}(u) = \mathbf{1}\{u \geq 0\}. \tag{2.7}$$

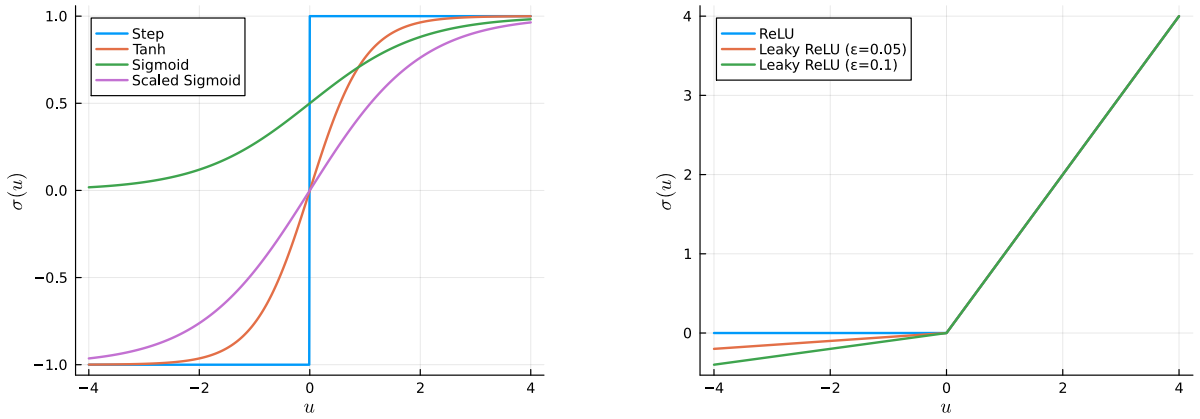
ReLU is **non-saturating** for positive inputs: its derivative is exactly 1 whenever $u > 0$, avoiding the vanishing gradient issue of sigmoid/tanh. It is also computationally cheap and was found empirically to train deep networks faster. The function is not differentiable at $u = 0$; by convention $\dot{\sigma}_{\text{ReLU}}(0) = 1$.

A network using only ReLU activations is a *piecewise linear* function of its input: the input space $\mathbb{R}^p$ is partitioned into polytopes, and on each polytope the network acts as a fixed affine map. This structure underlies many theoretical results on the expressivity of ReLU networks (see Section 2.4).

Leaky ReLU. ReLU neurons with $u < 0$ have zero gradient (*dying ReLU* problem): once a neuron's pre-activation becomes negative, it can stop updating. Leaky ReLU fixes this with a small slope $\varepsilon > 0$ for negative inputs:

$$\sigma_{\text{LeakyReLU}}(u) = \begin{cases} \varepsilon u & u < 0 \\ u & u \geq 0, \end{cases} \qquad \dot{\sigma}_{\text{LeakyReLU}}(u) = \begin{cases} \varepsilon & u < 0 \\ 1 & u \geq 0. \end{cases} \tag{2.8}$$

Setting $\varepsilon = 0$ recovers ReLU. A variant called *PReLU* (parametric ReLU) treats ε as a learnable parameter.



(a) Sigmoid (rescaled), tanh, and step function.

(b) ReLU and leaky ReLU variants.

Figure 2.2: Common scalar activation functions.

2.2.2 Key Properties

The following properties govern how well an activation function supports learning in deep networks.

- **Boundedness.** Sigmoid and tanh are bounded; ReLU and leaky ReLU are unbounded for positive inputs. Boundedness limits the magnitude of activations (which can help stability) but causes saturation.
- **Saturation.** An activation *saturates* when $|\dot{\sigma}(u)| \to 0$ as $|u| \to \infty$. Sigmoid and tanh saturate in both tails; ReLU saturates only for $u < 0$. Saturation reduces the gradient signal flowing backward through layers.
- **Lipschitz continuity.** A function σ is G-Lipschitz if $|\sigma(u) - \sigma(v)| \leq G|u - v|$ for all u, v. Sigmoid and tanh are 1-Lipschitz; ReLU is 1-Lipschitz. Lipschitz continuity of activations helps control how rapidly the network output can vary, which is important for stability.
- **Vanishing and exploding gradients.** Training a deep network requires gradients to be propagated backward through many layers. As we will see in Weeks 3–4, when gradients repeatedly pass through saturating or inappropriately scaled layers, they can shrink exponentially (*vanishing gradients*) or grow exponentially (*exploding gradients*). ReLU mitigates vanishing gradients since $\dot{\sigma}_{\text{ReLU}} = 1$ on the positive half-line, while proper weight initialisation (Section 2.5.3) helps control the exploding case.

Remark 2.2.1

For the **output layer**, the activation function is chosen to match the task rather than for trainability: sigmoid for binary classification, softmax for multi-class classification, and identity ($S^{[L]}(z) = z$) for regression. These were covered in Week 1.

Exercise 2.2.1

(a) Verify the identity $\sigma_{\text{Tanh}}(u) = 2\,\sigma_{\text{Sig}}(2u) - 1$. What does this imply about the relationship between networks using tanh and sigmoid activations in hidden layers?

(b) Show that σ_{ReLU} is 1-Lipschitz, *i.e.*, $|\sigma_{\text{ReLU}}(u) - \sigma_{\text{ReLU}}(v)| \leq |u - v|$ for all $u, v \in \mathbb{R}$. (*Hint*: consider separately the cases where both, one, or neither of u, v are positive.)

(c) Verify the derivative formula $\dot{\sigma}_{\text{Sig}}(u) = \sigma_{\text{Sig}}(u)(1 - \sigma_{\text{Sig}}(u))$ by direct differentiation. Use this to find the maximum value of $\dot{\sigma}_{\text{Sig}}$ and explain why this is relevant to the vanishing gradient problem.

2.3 Universal Approximation Theorem

A fundamental question in deep learning theory is: *what class of functions can a neural network represent?* The Universal Approximation Theorem (UAT) gives a striking answer: a feedforward network with a *single* hidden layer and a non-polynomial activation function can approximate any continuous function on a compact domain to arbitrary precision. This result justifies why neural networks are a sensible model class and underpins much of the theoretical foundation of deep learning.

2.3.1 A Constructive Example

Before stating the theorem, we illustrate the key idea constructively for scalar functions. Let $f^* : [\underline{l}, \bar{l}] \to \mathbb{R}$ be a continuous function, and suppose we evaluate it at r points $\underline{l} \leq u_1 < u_2 < \cdots < u_r \leq \bar{l}$ with values $v_i = f^*(u_i)$.

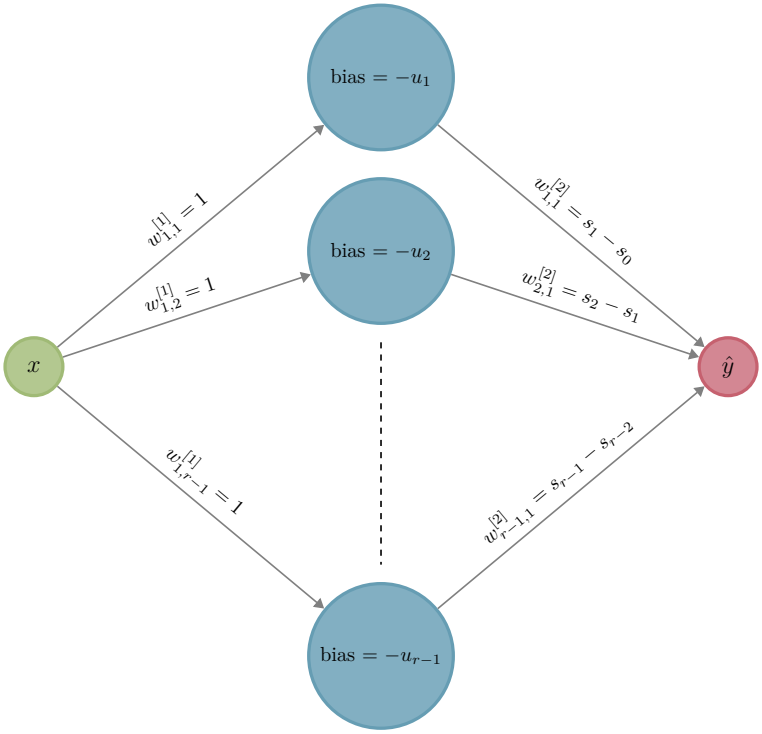
We construct a single hidden layer network with $N_0 = 1$, $N_1 = r - 1$, $N_2 = 1$, ReLU hidden activations, and identity output activation. Set:

$$W^{[1]} = \mathbf{1}_{(r-1)\times 1}, \quad b_i^{[1]} = -u_i \quad (i = 1, \ldots, r-1),$$

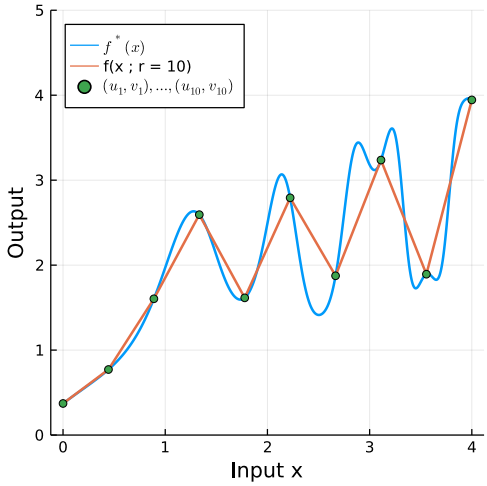
$$w_{1,i}^{[2]} = s_i - s_{i-1}, \quad b^{[2]} = v_1,$$

where $s_0 = 0$ and $s_i = \dfrac{v_{i+1} - v_i}{u_{i+1} - u_i}$ is the slope on the interval $[u_i, u_{i+1}]$.

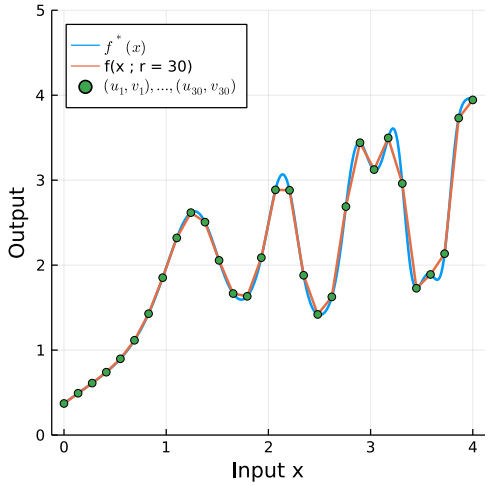
The i-th hidden neuron computes $\sigma_{\text{ReLU}}(u - u_i)$, which is a shifted ReLU that "switches on" at $u = u_i$. The output layer takes a signed linear combination of these shifted ReLUs to match the slope s_i on each interval, cancelling the previous slope via the $s_i - s_{i-1}$ weights. The resulting network satisfies $f_\theta(u_i) = f^*(u_i)$ for all i and is piecewise linear between sample points.



(a) Network architecture for the piecewise linear construction.



(b) $r = 10$ sample points.



(c) $r = 30$ sample points.

Figure 2.3: Approximating $f^*(x) = \cos\big(2\cos(x^2) + \frac{1}{4}(x-1)^2\big) + \frac{x}{2} + 1$ on $[0, 4]$ with a single hidden layer ReLU network. As the number of sample points r increases, the approximation improves.

As $r \to \infty$ (and the mesh $\max_i(u_{i+1} - u_i) \to 0$), the piecewise linear interpolant converges uniformly to f^* on $[\underline{l}, \bar{l}]$ for any continuous f^*, which follows from the uniform continuity of continuous functions on compact sets. This construction is one of many that can be used to prove the general

result below.

2.3.2 The Theorem

Theorem 2.3.1 (Universal Approximation, Cybenko 1989; Hornik et al. 1991)

Let $\mathcal{K} \subseteq \mathbb{R}^p$ be compact and let $f^* : \mathcal{K} \to \mathbb{R}^q$ be continuous. For any *non-polynomial* activation function $\sigma : \mathbb{R} \to \mathbb{R}$ and any $\varepsilon > 0$, there exists $N_1 \in \mathbb{N}$ and parameters $W^{[1]} \in \mathbb{R}^{N_1 \times p}$, $b^{[1]} \in \mathbb{R}^{N_1}$, $W^{[2]} \in \mathbb{R}^{q \times N_1}$ such that the two-layer network

$$f_\theta(x) = W^{[2]}\, S^{[1]}\big(W^{[1]}x + b^{[1]}\big)$$

satisfies $\|f_\theta(x) - f^*(x)\| < \varepsilon$ for all $x \in \mathcal{K}$.

The activation functions σ_{Sig}, σ_{Tanh}, and σ_{ReLU} are all non-polynomial and hence all valid for this result.

Remark 2.3.1

Theorem 2.3.2 is an *existence* result: it guarantees that approximating parameters exist, but gives no recipe for finding them. In practice, parameters are found by minimising the training loss via gradient-based optimisation (Weeks 3–4). The theorem also says nothing about how large N_1 needs to be — for a complicated f^* and small ε, the required width can be enormous.

Remark 2.3.2

The original proof of Cybenko (1989)[a] for sigmoid activations is a density argument from functional analysis (using the Hahn–Banach theorem). Hornik (1991)[b] subsequently extended the result to any non-polynomial activation, including ReLU. See the original papers for the technical details.

[a] G. Cybenko. *Approximation by superpositions of a sigmoidal function.* Mathematics of Control, Signals and Systems, 2(4):303–314, 1989.

[b] K. Hornik. *Approximation capabilities of multilayer feedforward networks.* Neural Networks, 4(2):251–257, 1991.

2.3.3 Implications and Limitations

- **Width vs. approximation quality.** A wider network (larger N_1) can approximate a given f^* to higher precision. For a fixed width, there will generally be a floor on the achievable error.
- **Non-polynomial activation is necessary.** If σ is a polynomial of degree d, then f_θ is also a polynomial and cannot approximate all continuous functions (e.g. it cannot approximate a function that oscillates more than d times).

- **Depth is not required for universality — but helps in practice.** The UAT shows depth $L = 2$ suffices. However, representing a function with a fixed-width deep network can require exponentially fewer parameters than a single hidden layer; this is the subject of Section 2.4.

Remark 2.3.3

The UAT guarantees approximation of *continuous* functions on *compact* domains, but says nothing about the ease of *learning* those parameters from data. A network may theoretically be able to represent f^*, yet training might converge to a poor solution or require impractically large amounts of data. Bridging the gap between representational capacity and learning is an active area of research.

2.4 Expressivity: Depth vs. Width

The UAT tells us that a network with a *single* hidden layer can approximate any continuous function — so why go deeper? The answer is *efficiency*: for many practically relevant function classes, achieving a target approximation error with a shallow network requires a width that grows exponentially in the problem complexity, whereas a deep network achieves the same accuracy with far fewer parameters. This section makes that intuition precise.

2.4.1 Depth Collapses Without Non-Linearity

A first observation is that depth alone buys nothing if the activation functions are linear. If $S^{[\ell]}$ is the identity at every hidden layer, then the network collapses to a single affine map regardless of how many layers it has:

$$f_\theta(x) = S^{[L]}\big(\tilde{W}x + \tilde{b}\big), \qquad \tilde{W} = W^{[L]} \cdots W^{[1]}, \quad \tilde{b} = \sum_{\ell=1}^{L} \Big(\prod_{\tilde{\ell}=\ell+1}^{L} W^{[\tilde{\ell}]} \Big) b^{[\ell]}. \tag{2.9}$$

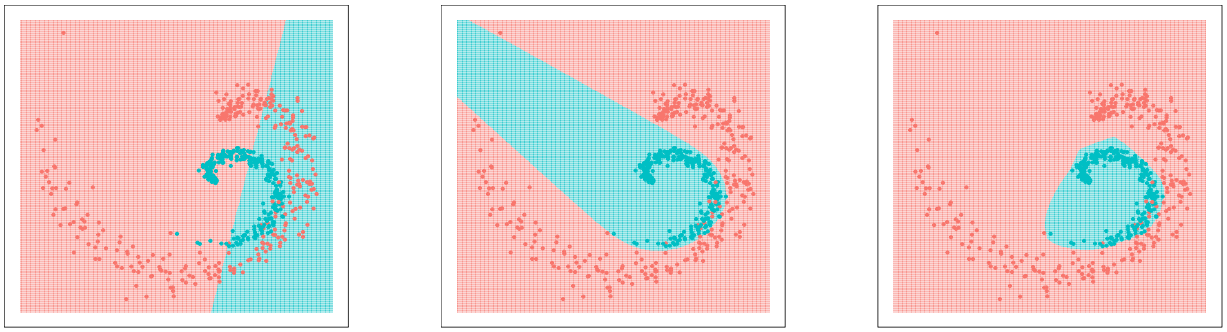
The product $\tilde{W}$ is just a single matrix and $\tilde{b}$ is just a single vector — the whole network is equivalent to $L = 1$. **All expressive power of a deep network comes from the composition of non-linear activations.**

Exercise 2.4.1

Verify the identity-collapse formula (2.9) for the case $L = 3$ by expanding the composition $W^{[3]}(W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}) + b^{[3]}$ directly, and confirm that your $\tilde{b}$ matches the closed-form expression in (2.9).

2.4.2 A Single Hidden Layer Can Approximate Interactions

Despite the eventual limitation of shallow networks, a single hidden layer already has significant expressive power beyond linear models. Adding hidden neurons enables *non-linear decision boundaries* that cannot be obtained by logistic or softmax regression alone.



(a) Logistic regression ($L = 1$). (b) One hidden layer, $N_1 = 4$. (c) One hidden layer, $N_1 = 10$.

Figure 2.4: Binary classification with $x \in \mathbb{R}^2$. A single hidden layer progressively refines the decision boundary as the width increases, going well beyond the linear boundary of logistic regression.

Multiplication gate. One useful measure of expressivity is the ability to approximate *pairwise interactions* $x_1 x_2$. Assuming the hidden-layer activation σ satisfies $\ddot{\sigma}(0) \neq 0$ (true for sigmoid and tanh, but *not* ReLU, which is piecewise linear), a single hidden layer with only $N_1 = 4$ neurons can construct a *multiplication gate*: a network $f_\theta : \mathbb{R}^2 \to \mathbb{R}$ with no bias terms and weight matrices

$$W^{[1]} = \begin{bmatrix} \lambda & \lambda \\ -\lambda & -\lambda \\ \lambda & -\lambda \\ -\lambda & \lambda \end{bmatrix}, \qquad W^{[2]} = \begin{bmatrix} \mu & \mu & -\mu & -\mu \end{bmatrix}, \quad \mu = \frac{1}{4\lambda^2 \ddot{\sigma}(0)},$$

satisfies $f_\theta(x_1, x_2) \approx x_1 x_2$. To see this, expand f_θ using the hidden-layer activations and apply a Taylor expansion $\sigma(u) = \sigma(0) + \dot{\sigma}(0)u + \ddot{\sigma}(0)\frac{u^2}{2} + O(u^3)$ to obtain

$$f_\theta(x_1, x_2) = x_1 x_2 \Big(1 + O\big(\lambda(x_1^2 + x_2^2)\big) \Big).$$

As $\lambda \to 0$ the error vanishes, so the approximation can be made arbitrarily precise. The schematic of this network is shown in Figure 2.5.

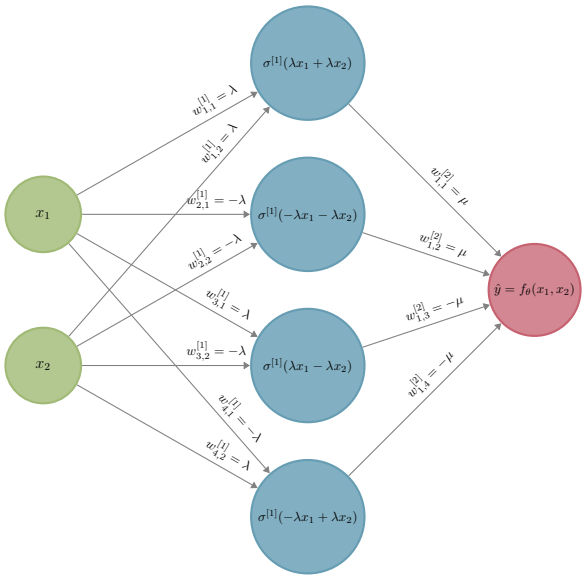


Figure 2.5: A four-neuron single hidden layer network implementing a multiplication gate: $\hat{y} \approx x_1 x_2$.

2.4.3 The Depth Advantage: A Sawtooth Construction

The multiplication-gate argument above shows that one hidden layer can already approximate complex functions, but with width that may scale unfavourably. Telgarsky (2016)[1] gives a much cleaner example of why *depth* matters: a family of *sawtooth* functions that a deep ReLU network represents with $O(L)$ neurons, but that any single-hidden-layer ReLU network would need $\Omega(2^L)$ neurons to match.

A triangle as one hidden layer. Consider the triangle function

$$g(x) \;=\; 2\,\sigma_{\mathrm{ReLU}}(x) \;-\; 4\,\sigma_{\mathrm{ReLU}}\big(x - \tfrac{1}{2}\big) \;+\; 2\,\sigma_{\mathrm{ReLU}}(x - 1). \tag{2.10}$$

Direct evaluation gives

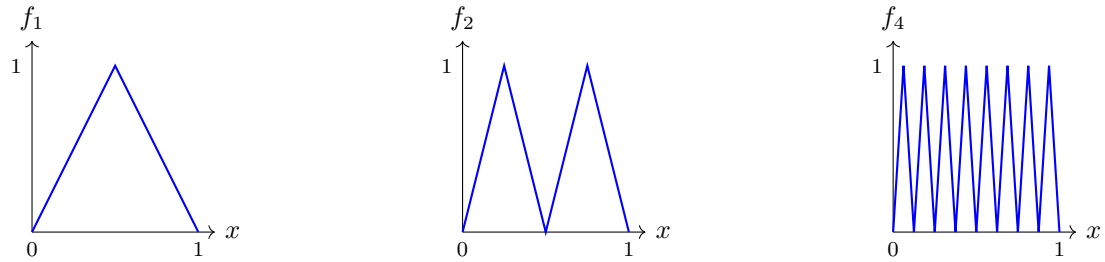
$$g(x) \;=\; \begin{cases} 2x & 0 \le x \le 1/2 \\ 2(1-x) & 1/2 \le x \le 1 \end{cases}, \qquad g(0) = g(1) = 0, \quad g(\tfrac{1}{2}) = 1.$$

This is exactly a depth-2 ReLU network with one input, hidden width 3, and one output — about 10 trainable parameters.

[1] M. Telgarsky. *Benefits of depth in neural networks.* In Conference on Learning Theory (COLT), PMLR 49:1517–1539, 2016.

Composing L triangles. Now stack L such layers and define $f_L = g \circ g \circ \cdots \circ g$ (L-fold composition). This is a depth-L ReLU network of constant width 3, with $12L - 14$ trainable parameters in total — so the parameter count is $O(L)$.

A short induction shows that f_L is a piecewise-linear function on $[0, 1]$ with 2^{L-1} "teeth" (i.e. 2^L linear pieces), shown in Figure 2.6. Each new composition takes every existing tooth and splits it into two.



(a) $L = 1$: 1 tooth. (b) $L = 2$: 2 teeth. (c) $L = 4$: 8 teeth.

Figure 2.6: Composing the triangle function g doubles the number of teeth at each step. A depth-L ReLU network of width 3 realises f_L, a piecewise-linear function with 2^{L-1} teeth on $[0, 1]$.

Shallow networks need exponential width. Now try to match f_L with a single-hidden-layer ReLU network

$$\hat{f}(x) \;=\; \sum_{i=1}^{N_1} \alpha_i \, \sigma_{\mathrm{ReLU}}(w_i x + b_i) + c.$$

Each ReLU neuron contributes at most one slope-change to $\hat{f}$ (each $\sigma_{\mathrm{ReLU}}(w_i x + b_i)$ is zero on one side of the point $x = -b_i/w_i$ and linear on the other, so it changes slope exactly once), and a sum of such functions has slope-changes only where one of the summands does. Hence $\hat{f}$ is piecewise-linear with at most $N_1 + 1$ linear pieces. Since f_L has 2^L linear pieces, matching it requires

$$N_1 \;\geq\; 2^L - 1, \tag{2.11}$$

which is exponential in the depth L that the deep network used.

Example 2.4.1

For $L = 10$:

- Deep: 10 hidden layers of width 3 $\Rightarrow$ about 100 parameters.
- Shallow (depth 2): width $\geq 2^{10} - 1 = 1023$ neurons $\Rightarrow$ about 3000 parameters.

For $L = 20$ the shallow network would need over a million neurons. The gap is exponential in L.

Remark 2.4.1

This is the canonical *depth-separation* result: a family of functions expressible by depth-L networks of size $O(L)$ that provably require size $\Omega(2^L)$ from any depth-2 network. The construction above is due to Telgarsky (2016); Eldan & Shamir (2016)[a] give an analogous separation in a different setting. The UAT says shallow networks *can* approximate any continuous function, but functions with rich hierarchical structure (oscillations, repeated compositions) may require an exponentially larger shallow network than a deep one.

[a] R. Eldan and O. Shamir. *The power of depth for feedforward neural networks.* In Conference on Learning Theory (COLT), PMLR 49:907–940, 2016.

2.5 Training Neural Networks

Sections 2.3–2.4 addressed what neural networks *can* represent. We now turn to how the parameters $\theta = \{W^{[\ell]}, b^{[\ell]}\}_{\ell=1}^{L}$ are *found* from data. The full story — automatic differentiation, convergence guarantees, adaptive optimisers — is the subject of Weeks 3–4. Here we lay the conceptual foundation: the training objective, the backpropagation algorithm for computing gradients, and weight initialisation.

2.5.1 Training Objective

Given a dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$, training a neural network is an instance of empirical risk minimisation (Week 1):

$$\hat{\theta} = \arg\min_{\theta} \; C(\theta), \qquad C(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(y^{(i)},\, f_\theta(x^{(i)})\big). \tag{2.12}$$

The choice of loss ℓ matches the output layer activation:

- **Regression** (identity output): squared loss $\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$.
- **Binary classification** (sigmoid output): binary cross-entropy $\ell(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$.
- **Multi-class classification** (softmax output): categorical cross-entropy,

$$\ell(y, \hat{y}) = -\textstyle\sum_k y_k \log \hat{y}_k.$$

Unlike linear or logistic regression, the objective $C(\theta)$ is non-convex in θ due to the composition of non-linear layers. Gradient-based optimisation (gradient descent and its variants) is therefore used to find a local minimum.

2.5.2 Backpropagation

Computing $\nabla_\theta C(\theta)$ requires differentiating through the composition (2.3). The **backpropagation algorithm** does this efficiently by a single backward pass through the network using the chain rule. It is a special case of reverse-mode automatic differentiation (covered in detail in Weeks 3–4).

Parameter updates. Once the gradient $\nabla_\theta C(\theta)$ is available (computed by backpropagation), parameters are updated by **gradient descent**, exactly as in Week 1:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta C\big(\theta^{(t)}\big), \tag{2.13}$$

where $\alpha > 0$ is the learning rate. The only difference from Week 1 is that θ now collects all weight matrices and bias vectors $\{W^{[\ell]}, b^{[\ell]}\}_{\ell=1}^{L}$ rather than a single parameter vector.

In practice, for large datasets the full gradient is too expensive to compute at every step. It is replaced by a cheap unbiased estimate computed on a randomly sampled mini-batch (**stochastic gradient descent**, SGD), and adaptive variants such as RMSProp and Adam adjust the per-parameter learning rate on the fly. These methods are the standard tools of modern deep learning and are covered in detail in Weeks 3–4.

2.5.3 Weight Initialisation

A key practical decision before training is how to initialise the parameter values $\theta^{(0)}$. Setting all weights to zero is problematic: every neuron in a layer would compute the same gradient and remain identical throughout training (*symmetry breaking* fails). Initialising with large random values can cause exploding gradients, while very small values cause vanishing gradients.

Variance analysis. Consider a single hidden layer with pre-activation

$$z_i^{[\ell]} = \sum_{j=1}^{N_{\ell-1}} w_{i,j}^{[\ell]} a_j^{[\ell-1]} + b_i^{[\ell]}.$$

Assuming the weights $w_{i,j}^{[\ell]}$ and activations $a_j^{[\ell-1]}$ are independent, zero-mean, with $\mathrm{Var}(w_{i,j}^{[\ell]}) = \breve{w}_\ell$ (and biases initialised to zero, so they do not contribute to the variance), the variance of the pre-activation propagates as:

$$\mathrm{Var}(z_i^{[\ell]}) = N_{\ell-1}\, \breve{w}_\ell\, \mathrm{Var}(a_j^{[\ell-1]}). \tag{2.14}$$

For the variance to remain stable across layers we want $N_{\ell-1}\, \breve{w}_\ell = 1$, giving the **Xavier (Glorot) initialisation**:

$$\mathrm{Var}(w_{i,j}^{[\ell]}) = \frac{1}{N_{\ell-1}}. \tag{2.15}$$

He initialisation. For ReLU activations, only half the neurons are active on average (the positive half), effectively halving the signal variance. He et al. (2015)[2] account for this by doubling the variance:

$$\mathrm{Var}(w_{i,j}^{[\ell]}) = \frac{2}{N_{\ell-1}}. \tag{2.16}$$

In both schemes, weights are drawn i.i.d. from a zero-mean distribution (typically Gaussian or uniform) with the specified variance, and biases are initialised to zero.

Remark 2.5.1

Without proper initialisation, training very deep networks was historically very difficult. The practical success of Xavier initialisation (Glorot & Bengio, 2010)[a] and He initialisation (He et al., 2015) was a key factor in making deep networks trainable. Batch normalisation, discussed in Section 2.6, provides a further mechanism to control activation variance dynamically during training.

[a] X. Glorot and Y. Bengio. *Understanding the difficulty of training deep feedforward neural networks.* In International Conference on Artificial Intelligence and Statistics (AISTATS), 9:249–256, 2010.

Exercise 2.5.1

(a) Derive the variance propagation formula (2.14) rigorously. Starting from $z_i^{[\ell]} = \sum_{j=1}^{N_{\ell-1}} w_{i,j}^{[\ell]} a_j^{[\ell-1]}$ (ignoring bias), and assuming $w_{i,j}^{[\ell]}$ and $a_j^{[\ell-1]}$ are independent with zero mean and $\mathrm{Var}(w_{i,j}^{[\ell]}) = \breve{w}_\ell$, show that $\mathrm{Var}(z_i^{[\ell]}) = N_{\ell-1}\, \breve{w}_\ell\, \mathrm{Var}(a_j^{[\ell-1]})$. (*Hint*: use the identity $\mathrm{Var}(XY) = \mathrm{Var}(X)\mathrm{Var}(Y)$ for independent zero-mean variables; then use independence across j to expand $\mathrm{Var}\big(\sum_j w_{i,j}^{[\ell]} a_j^{[\ell-1]}\big) = \sum_j \mathrm{Var}\big(w_{i,j}^{[\ell]} a_j^{[\ell-1]}\big)$.)

(b) For a network with $L = 4$ layers and widths $(N_0, N_1, N_2, N_3, N_4) = (10, 8, 6, 4, 1)$ using Xavier initialisation (2.15), compute $\mathrm{Var}(w_{i,j}^{[\ell]})$ for each layer $\ell = 1, \ldots, 4$.

2.6 Regularisation and Generalisation

The UAT guarantees that a sufficiently wide or deep network can *fit* any training set perfectly. Without restraint, this leads to overfitting: the model memorises the training data and fails to generalise. Recall from Week 1 the bias–variance tradeoff: very expressive models have low bias but high variance. Regularisation techniques use a variety of mechanisms — penalising the loss, injecting noise, normalising activations, or limiting training time — to control model complexity and improve generalisation.

[2] K. He, X. Zhang, S. Ren, and J. Sun. *Delving deep into rectifiers: surpassing human-level performance on ImageNet classification.* In International Conference on Computer Vision (ICCV), pp. 1026–1034, 2015.

2.6.1 Weight Decay (L2 Regularisation)

The most direct approach is to augment the training objective with an L2 penalty on the weights:

$$\tilde{C}(\theta) = C(\theta) + \frac{\lambda}{2}\|\theta\|^2, \tag{2.17}$$

where $\lambda > 0$ controls the regularisation strength. Typically, the penalty is applied only to weight matrices $W^{[\ell]}$ and not to bias vectors.

A gradient descent step on $\tilde{C}$ gives

$$\theta^{(t+1)} = (1 - \alpha\lambda)\,\theta^{(t)} - \alpha\,\nabla C(\theta^{(t)}). \tag{2.18}$$

The factor $(1 - \alpha\lambda)$ *shrinks* (decays) the weights at every iteration, independently of the gradient. This is the origin of the name **weight decay**. The effect is to keep weight magnitudes small, preventing the network from placing too much confidence in any single feature or pathway. Weight decay is used with all gradient-based optimisers, including Adam, by appending the multiplicative decay step to each update.

2.6.2 Dropout

Dropout is a regularisation technique specific to neural networks. During each training iteration, each neuron in layer ℓ is *randomly dropped* (set to zero) with probability $1 - p_{\text{keep}}^{[\ell]}$, independently for every neuron, every layer, and every iteration.

Forward pass with dropout. Neuron j in layer $\ell+1$ only receives input from the *kept* neurons of layer ℓ:

$$a_j^{[\ell+1]} = \sigma\Big(b_j^{[\ell+1]} + \sum_{i\,\text{kept}} w_{i,j}^{[\ell+1]} a_i^{[\ell]}\Big). \tag{2.19}$$

In the backward pass, gradients are not propagated through dropped neurons, so their incoming weights are not updated in that iteration.

Inverted dropout. To ensure the expected magnitude of activations is consistent between training and test time, the kept activations are scaled by $1/p_{\text{keep}}^{[\ell]}$ during training:

$$a_j^{[\ell+1]} = \sigma\Big(b_j^{[\ell+1]} + \sum_{i\,\text{kept}} \frac{1}{p_{\text{keep}}^{[\ell]}} w_{i,j}^{[\ell+1]} a_i^{[\ell]}\Big). \tag{2.20}$$

This *inverted dropout* formulation means the trained weight matrices can be used unchanged at test time, with no dropout applied.

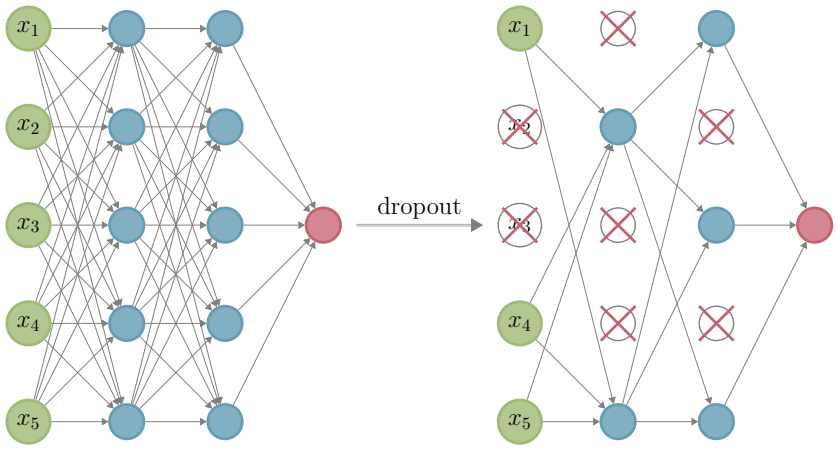


Figure 2.7: Dropout during training. For each iteration, a random subset of neurons (shown crossed out) is dropped, effectively training a different sub-network at each step.

Dropout as ensemble learning. A useful interpretation is that each training iteration trains a different *sub-network* (determined by which neurons survive). The final production model can be viewed as an implicit ensemble over all the sub-networks encountered during training. Ensemble averaging reduces variance without increasing bias, which explains dropout's regularising effect.

Remark 2.6.1

Typical keep probabilities are $p_{\text{keep}}^{[\ell]} \in [0.5, 0.8]$ for hidden layers and $p_{\text{keep}}^{[0]} \approx 0.8$–$1.0$ for the input layer. The output layer is never dropped. Dropout is usually disabled during evaluation.

2.6.3 Batch Normalisation

Batch normalisation normalises the pre-activations $z^{[\ell]}$ within each mini-batch, stabilising the distribution of inputs to each layer as training progresses. This reduces the sensitivity to weight initialisation and mitigates vanishing/exploding gradients.

Normalisation step. For each neuron j in layer ℓ and mini-batch of size n_b, compute the batch mean and standard deviation:

$$\hat{\mu}_j^{[\ell]} = \frac{1}{n_b}\sum_{i=1}^{n_b} z_j^{[\ell](i)}, \qquad \hat{\sigma}_j^{[\ell]} = \sqrt{\frac{1}{n_b}\sum_{i=1}^{n_b}\big(z_j^{[\ell](i)} - \hat{\mu}_j^{[\ell]}\big)^2}, \tag{2.21}$$

then normalise:

$$\bar{z}_j^{[\ell](i)} = \frac{z_j^{[\ell](i)} - \hat{\mu}_j^{[\ell]}}{\sqrt{(\hat{\sigma}_j^{[\ell]})^2 + \varepsilon}}, \tag{2.22}$$

where $\varepsilon > 0$ prevents division by zero.

Scale and shift. A fixed normalisation to zero mean and unit variance is too restrictive — the network should be able to learn any desired distribution. Two *trainable* parameters $\gamma_j^{[\ell]}$ and $\beta_j^{[\ell]}$ (initialised at 1 and 0) restore this flexibility:

$$\tilde{z}_j^{[\ell](i)} = \gamma_j^{[\ell]}\, \bar{z}_j^{[\ell](i)} + \beta_j^{[\ell]}. \tag{2.23}$$

The transformed pre-activation $\tilde{z}_j^{[\ell]}$ is used in place of $z_j^{[\ell]}$ throughout the forward and backward passes. The parameters $\gamma_j^{[\ell]}$ and $\beta_j^{[\ell]}$ are updated by backpropagation alongside the network weights.

Test time. At test time, batch statistics cannot be computed (no mini-batch). Instead, running averages $\hat{\bar{\mu}}_j^{[\ell]}$ and $\hat{\bar{\sigma}}_j^{[\ell]}$, collected during training via exponential smoothing, are used:

$$\tilde{z}_j^{[\ell]} = \gamma_j^{[\ell]} \frac{z_j^{[\ell]} - \hat{\bar{\mu}}_j^{[\ell]}}{\sqrt{(\hat{\bar{\sigma}}_j^{[\ell]})^2 + \varepsilon}} + \beta_j^{[\ell]}. \tag{2.24}$$

Note that (2.24) is an affine transformation of $z^{[\ell]}$, so in principle the batch normalisation can be folded directly into the weight matrices and bias vectors at deployment.

2.6.4 Early Stopping

Early stopping is a simple but effective technique: monitor the loss on a held-out validation set during training and stop once validation loss begins to increase, even if training loss is still decreasing. This prevents the model from overfitting by halting the optimisation before it can memorise training noise.

Remark 2.6.2

It can be shown that for linear models with gradient descent, early stopping is approximately equivalent to L2 regularisation. The regularisation strength is determined by the number of training steps: fewer steps $\approx$ stronger regularisation. For non-linear networks the equivalence is only approximate, but early stopping remains one of the most widely used practical tools for controlling overfitting.

Exercise 2.6.1

(a) Show directly that the weight decay update (2.18) is exactly one step of gradient descent on the regularised objective $\tilde{C}(\theta) = C(\theta) + \frac{\lambda}{2}\|\theta\|^2$ with learning rate α.

(b) For dropout with keep probability $p_{\text{keep}}^{[\ell]}$, show that the expected value of the *pre-activation* $z_j^{[\ell+1]} = b_j^{[\ell+1]} + \sum_{i\,\text{kept}} w_{i,j}^{[\ell+1]} a_i^{[\ell]}$ in the training forward pass equals $p_{\text{keep}}^{[\ell]}$ times the value without dropout. Then verify that inverted dropout (2.20) corrects for this, making the expected training and test pre-activations equal.

Remark 2.6.3

In practice, even without any explicit regularisation term, gradient descent exhibits *implicit regularisation*: the optimisation trajectory itself tends to favour solutions with small norm. For overparameterised least squares started from $\theta^{(0)} = 0$, gradient descent converges to the minimum-L_2-norm interpolating solution. For general neural networks the precise characterisation remains an active research area; Week 4 will touch on a related effect, where the noise in stochastic gradient descent biases training toward *flatter* minima that tend to generalise better.